NAME: KEERTHIKA S **ROLL NO: 231901024**

TITLE: SETTING UP HYPERLEDGER FABRIC DEVELOPMENT ENVIRONMENT

1. AIM

To install and configure a complete Hyperledger Fabric development environment on Ubuntu Linux — including all prerequisite software, the Fabric sample network, platform-specific binaries, and Docker images — and to verify the setup by bringing up the Fabric “test-network” successfully.

2. TOOLS / SOFTWARE REQUIRED

- Ubuntu 20.04 LTS or 22.04 LTS (64-bit)
- Git – version control tool to clone Fabric repositories
- cURL – to download the Fabric bootstrap script
- Docker Engine & Docker Compose – to run Fabric peer, orderer and CA containers
- Go (Golang) – version 1.20 or later, for chaincode and Fabric binaries
- Node.js and npm – for the Fabric SDK and application development
- Python 3 – required by some Fabric build/legacy scripts
- Hyperledger Fabric Samples, Binaries and Docker Images (fetched via bootstrap script)

3. PROCEDURE

The environment is set up in the following sequence: update the system, install prerequisite packages, install and configure Docker, install Go and Node.js, download the Fabric samples/binaries/images using the official bootstrap script, and finally bring up the test network to validate the installation.

Step 1: Update system packages

```
sudo apt update && sudo apt upgrade -y
```

Step 2: Install Git, cURL and build essentials

```
sudo apt install -y git curl build-essential
```

Step 3: Install Docker Engine and Docker Compose

```
sudo apt install -y docker.io docker-compose
sudo systemctl enable docker
sudo systemctl start docker
sudo usermod -aG docker $USER
newgrp docker
```

Step 4: Verify Docker installation

```
docker --version
docker-compose --version
```

Step 5: Install Go (Golang)

```
wget https://go.dev/dl/go1.21.0.linux-amd64.tar.gz
sudo tar -C /usr/local -xzf go1.21.0.linux-amd64.tar.gz
echo 'export PATH=$PATH:/usr/local/go/bin' >> ~/.bashrc
source ~/.bashrc
go version
```

Step 6: Install Node.js and npm

```
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt install -y nodejs
node -v && npm -v
```

Step 7: Download and run the Fabric bootstrap script

```
mkdir -p ~/fabric && cd ~/fabric
curl -sSLO
https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/bootstrap.sh &&
chmod +x bootstrap.sh
./bootstrap.sh
```

Step 8: Verify downloaded binaries and Docker images

```
cd fabric-samples/bin && ls
docker images | grep hyperledger
```

Step 9: Bring up the Fabric test network

```
cd ~/fabric/fabric-samples/test-network
./network.sh up createChannel -c mychannel -ca
```

Step 10: Verify running containers

```
docker ps
```

4. SUMMARY OF KEY COMMANDS

Purpose	Command
Update packages	`sudo apt update && sudo apt upgrade -y`
Install Docker	`sudo apt install -y docker.io docker-compose`
Check Docker version	`docker --version`
Install Go	`sudo tar -C /usr/local -xzf go1.21.0.linux-amd64.tar.gz`
Install Node.js	`sudo apt install -y nodejs`
Run bootstrap script	`./bootstrap.sh`
Start test network	`./network.sh up createChannel -c mychannel -ca`
List running containers	`docker ps`
Bring network down	`./network.sh down`

5. OUTPUT

The following terminal outputs illustrate the expected result at each key verification stage of the setup. Replace these blocks with your own captured screenshots if required.

Output 1: Docker installation check

```
$ docker --version
Docker version 24.0.5, build ced0996

$ docker-compose --version
Docker Compose version v2.20.2
```

Fig 5.1 – Docker and Docker Compose successfully installed.

Output 2: Go and Node.js version check

```
$ go version
go version go1.21.0 linux/amd64

$ node -v && npm -v
v18.17.1
9.6.7
```

Fig 5.2 – Go and Node.js installed and configured on PATH.

Output 3: Fabric binaries and Docker images

```
$ ls fabric-samples/bin
configtxgen configtxlator cryptogen discover fabric-ca-client
fabric-ca-server ledgerutil orderer osnadmin peer

$ docker images | grep hyperledger
hyperledger/fabric-peer 2.5 3a1f9c2b7e4d 2 weeks ago 58.3MB
hyperledger/fabric-orderer 2.5 9b7e4d3a1f2c 2 weeks ago 45.1MB
hyperledger/fabric-ca 1.5 7e4d3a1f9c2b 3 weeks ago 129MB
```

Fig 5.3 – Fabric binaries and required Docker images downloaded successfully.

Output 4: Test network brought up successfully

```
$ ./network.sh up createChannel -c mychannel -ca
Creating network "fabric_test" with the default driver
Creating orderer.example.com ... done
Creating peer0.org1.example.com ... done
Creating peer0.org2.example.com ... done
Creating ca_org1 ... done
Creating ca_org2 ... done
Channel 'mychannel' created
Joining org1 peer to the channel...
Joining org2 peer to the channel...
```

Fig 5.4 – Test network containers created and channel 'mychannel' set up.

Output 5: Verifying running containers

```
$ docker ps
CONTAINER ID IMAGE STATUS NAMES
5f3a1b2c4d6e hyperledger/fabric-peer Up 2 minutes peer0.org1.example.com
8e7d6c5b4a3f hyperledger/fabric-peer Up 2 minutes peer0.org2.example.com
2c1b9a8d7e6f hyperledger/fabric-orderer Up 2 minutes orderer.example.com
9d8c7b6a5e4f hyperledger/fabric-ca Up 2 minutes ca_org1
4e3d2c1b9a8f hyperledger/fabric-ca Up 2 minutes ca_org2
```

Fig 5.5 – All Fabric network containers are up and running, confirming successful setup.

6. RESULT

The Hyperledger Fabric development environment was successfully set up on Ubuntu Linux. All prerequisite tools (Git, Docker, Go, Node.js) were installed and verified, the Fabric samples, binaries and Docker images were downloaded using the bootstrap script, and the Fabric test network was brought up successfully with the required peer, orderer and CA containers running, confirming that the environment is ready for chaincode development and deployment.